Exact Data Match (EDM) Configuration — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Exact Data Match (EDM) in Microsoft Purview 4

The problem . . . 4

The solution . . . 4

What it delivers . . . 4

Business outcomes . . . 4

Boundaries (so scope is clear) . . . 5

Solution Overview — Exact Data Match (EDM) in Microsoft Purview 5

What this solution delivers . . . 5

How EDM keeps your data safe — read this first . . . 6

The EDM model — how the pieces fit . . . 6

The standard deployment model . . . 7

What's in the box . . . 7

2. Requirements — Exact Data Match (EDM) in Microsoft Purview 8

R1 — EDM-capable licensing (E5 / E5 Compliance) . . . 8

R2 — EDM authoring roles (least privilege) . . . 8

R3 — EDM_DataUploaders authorization (upload account) . . . 9

R4 — The EDM Upload Agent installed for the correct cloud . . . 9

R5 — The sensitive-data source and schema design (with the data owner) . . . 10

R6 — A secure host and secret handling for hashing (salt + plaintext) . . . 10

R7 — In-scope workloads and a consuming policy design . . . 10

R8 — Pilot scope, change control, and the refresh plan . . . 11

Global Requirements — Microsoft Purview Solutions 12

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation — Exact Data Match (EDM) in Microsoft Purview 14

Phase 0 — Assess (confirm you can build safely) . . . 14

Phase 1 — Design (agree it with the data owner, then name everything) . . . 15

Phase 2 — Create the EDM schema and SIT . . . 15

Phase 3 — Export, hash, and upload the sensitive data (plaintext stays local) . . . 16

Phase 4 — Validate the SIT matches your data (before any policy) . . . 17

Phase 5 — Build the DLP enforcement in SIMULATION/TEST first . . . 17

Phase 6 — Pilot, validate, then expand (enforce, then widen) . . . 18
5. Testing & Validation — Exact Data Match (EDM) in Microsoft Purview 19
Test A — Exact match on real data (it detects what it should) . . . 19
Test B — Exactness (it does NOT match look-alikes) — the defining EDM gate . . . 20
Test C — Policy enforcement (the control is operable) . . . 20
Test D — Data store health & refresh (matching stays accurate) . . . 21
Simulation-first & rollback validation . . . 21
6. Troubleshooting — Exact Data Match (EDM) in Microsoft Purview 22
The EDM node / schema creation is missing or greyed out . . . 22
Upload fails with an authorization / access error . . . 23
`EdmUploadAgent.exe` errors on the data file . . . 23
The SIT doesn't match values you KNOW are in the data . . . 23
The SIT matches values that are NOT in the data (over-matching) . . . 24
Newly-added records don't match / removed records still match . . . 24
The DLP policy isn't acting on EDM matches . . . 24
`Test-DataClassification` / validation script returns nothing . . . 24
Government / sovereign-cloud specifics . . . 25
7. Best Practices — Exact Data Match (EDM) in Microsoft Purview 25
Protect the plaintext and the salt — non-negotiable . . . 25
Design the schema for exactness and performance . . . 25
Refresh the data — a stale EDM data store silently fails . . . 26
Simulate before you enforce — always . . . 26
Use EDM where pattern SITs over-fire — not everywhere . . . 26
Operations & governance . . . 27
Stay inside the boundary . . . 27
8. Deliverables Summary — Exact Data Match (EDM) in Microsoft Purview 27
Documentation . . . 27
PowerShell tools (5) — Security & Compliance PowerShell / Graph . . . 28
Definition of done . . . 29
Appendix A — PowerShell Scripts 31

Executive Summary — Exact Data Match (EDM) in Microsoft Purview

The problem

Pattern-based data classifiers guess. A built-in credit-card, SSN, or account-number detector fires on *anything shaped like* the sensitive value — including test data, a random number, or an employee's own details — so it over-matches, buries the triage team in false positives, and gets tuned down or turned off. The organization is left either **over-blocking legitimate work** or **under-protecting the real records** that matter — with no way to say "protect *these specific* customers, and only these."

The solution

Implement **Exact Data Match (EDM)** in Microsoft Purview, using only the Microsoft Purview portal, Security & Compliance PowerShell / Graph, and Microsoft's **EDM Upload Agent**. EDM matches on your **actual records** — the exact account numbers, IDs, and PII in your own database — and on nothing else. Crucially, it is **privacy-preserving**: the source data is **hashed and salted on a machine you control**, and **only the one-way hashes are uploaded** — the plaintext never leaves your environment. The resulting sensitive information type is used in a DLP policy that is **built and proven in SIMULATION first**, then tuned, then enforced to a pilot ring, then expanded — so exact protection goes live without disrupting the business.

What it delivers

- An **EDM schema** and **EDM sensitive information type** that match your real records exactly — an exact primary value plus supporting elements in proximity, at a defined confidence.
- A **privacy-preserving data pipeline** — hash-and-upload with the EDM Upload Agent, plaintext kept local and securely deleted, the salt held as a secret, and a **refresh schedule** so matching stays current.
- A **DLP policy** referencing the EDM SIT, proven in simulation and enforced through policy tips, override, or block.
- A **near-zero false-positive rate** — the defining EDM advantage: look-alike and not-in-data values are correctly ignored, so the control is trusted and can be safely enforced.
- **Five PowerShell tools** (readiness, schema helper, validation, reporting, scope helper) — read-only or `-WhatIf` by default, with **no credentials, salts, or record data in code**.

Business outcomes

- **Exact protection** — the right *specific* records are detected and protected, not everything that merely looks similar.
- **Trusted, low-noise control** — a near-zero false-positive rate is the difference between a control people rely on and one they ignore, and it makes enforcement defensible.

• **Privacy by design** — sensitive data is hashed locally; only hashes are uploaded; plaintext is

securely deleted — a posture you can show privacy and compliance.

• **Self-guided at a fraction of consulting cost** — the same method a specialist would use, documented

and tooled to run in-house.

Boundaries (so scope is clear)

EDM is licensing-dependent — it requires **Microsoft 365 E5 / E5 Compliance / Information Protection & Governance** (verify your tenant), and **Government clouds (GCC High / DoD) use a distinct Upload Agent and endpoints**. This solution provides the **technical configuration and the evidence**; *which* records are lawful to load, the **enforced action**, and the **minimisation and retention** of the loaded data are **compliance, privacy, and legal determinations**, captured alongside the delivered evidence.

Solution Overview — Exact Data Match (EDM) in Microsoft Purview

What this solution delivers

Pattern-based classifiers are guessers. A built-in **Credit Card Number** or **U.S. SSN** detector fires on *anything shaped like* a card number or an SSN — including test data, a colleague's own SSN, or a random nine-digit string — so it over-matches, floods triage with false positives, and gets tuned down or turned off. **Exact Data Match (EDM)** replaces the guess with certainty: it matches on **your actual records** — the exact account numbers, patient IDs, or customer rows in *your* database — and matches on nothing else. The result is a sensitive information type (SIT) with a **near-zero false-positive rate** that you can safely enforce.

This solution walks a competent-but-novice administrator through a **complete, repeatable EDM deployment** in Microsoft Purview:

1. **Design the schema** — define the structure of the sensitive data (the columns, which field is

the **primary/searchable element**, which are **supporting elements**, and the per-field configurable-match options).

1. **Prepare the sensitive-data source** — export the actual records to a delimited (`.csv`/`.tsv`)

file whose header row matches the schema.

1. **Hash and upload the data with the EDM Upload Agent** — the source records are **hashed and

salted on your own machine **by** `EdmUploadAgent.exe`**;** only the one-way hashes leave your environment **— the plaintext sensitive data is** never** uploaded to the service.

1. **Create the EDM sensitive information type** — a SIT that references the schema, requires an

exact match on the primary element (against the uploaded hashes) plus **supporting elements in proximity**, at a defined confidence.

1. **Wire it into a policy and pilot in SIMULATION** — use the EDM SIT in a **DLP policy** (and,

optionally, auto-labeling / retention) run in **simulation/test first**, validate, then enforce.

1. **Operate the refresh** — keep the hashed data current on a schedule, because a stale EDM data

store silently degrades matching.

Together these turn a **noisy, pattern-based guess** into an **exact, evidence-backed** detector: strong where it needs to be, silent everywhere else — configured, piloted, validated, and defensible.

How EDM keeps your data safe — read this first

EDM is **privacy-preserving by design**, and understanding why is central to deploying it correctly:

• **The plaintext never leaves your tenant boundary.** `EdmUploadAgent.exe` runs **on a machine you

control**, reads the source file locally, and computes a** salted one-way hash **of each cell. Only those hashes (plus the schema metadata) are uploaded. Microsoft's service stores** hashes, not values** — it cannot reconstruct the original records from them.

• **You own the salt.** You supply (or let the agent generate) a **salt** used in the hashing. Keep

it secret and consistent — it is part of what makes the hashes non-reversible and is required to re-hash matching data later.

• **The source data is real sensitive data — handle it as such.** The `.csv`/`.tsv` export contains

live records (account numbers, IDs, PII). Generate it under change control, keep it on an access-controlled host, hash it, and **securely delete the plaintext export** when done. **Which data is lawful to load, and its retention, are privacy/compliance/legal determinations** — this solution provides the technical method and the guardrails, not the determination (see `global-requirements-purview.md` §5).

> ***Deploy this with your data owners, privacy, and compliance stakeholders.*** *EDM loads a hashed copy of a real records set into a Microsoft service; the* ***applicability, minimisation, and retention of that data*** *are governance decisions for your organization.*

The EDM model — how the pieces fit

EDM has **four moving parts**. Get their relationship right and the rest is mechanical:

• **1. The EDM schema (the data store definition).** An XML/portal-defined structure listing the

fields (columns) in your sensitive data. Each field can be marked:

• **Searchable / primary element** — a field EDM builds a fast lookup index on; a document must

contain an **exact match to one of these** to be a candidate. At least one field must be searchable.

• **Configurable match** per field — **ignore case** and/or **ignore delimiters** (so a stored

`123-45-6789` still matches a document's `123456789`).

• **2. The sensitive information source table (your data).** A delimited file (`.csv` or `.tsv`)

whose **first row is the field names** (matching the schema) and each subsequent row is one record. The data store name ties this file to its schema.

- **3. The hashed & uploaded data.** `EdmUploadAgent.exe` hashes the source table locally and uploads

the hashes to the EDM service's **data store** for that schema. This is what documents are matched against — and it must be **refreshed** as the underlying data changes.

- **4. The EDM sensitive information type (the SIT / rule package).** The classifier that references

the schema and says: *"match when a document contains an exact value from the **primary element** index **and** N **supporting elements** within a **proximity window**, at confidence X."* This SIT then behaves like any other SIT — usable in DLP, auto-labeling, retention, Communication Compliance, eDiscovery, and Insider Risk.

> ***Two authoring paths.*** *The **Microsoft Purview portal** *Exact data match (EDM)* wizard creates the **schema and the SIT together** and is the recommended path. The equivalent **PowerShell/XML** path (`New-DlpEdmSchema`, `New-DlpSensitiveInformationTypeRulePackage`) is available for repeatable, source-controlled definitions. **The hashing/upload step is always the EDM Upload Agent** — there is no portal upload of plaintext. Portal paths and schema limits (field counts, row/size caps) **evolve and are licensing-dependent — verify against Microsoft Learn and your tenant** rather than assuming.*

The standard deployment model

Every step follows the same repeatable model, with an explicit **pass/fail gate** between phases:

Assess → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand (enforce).**

- **Assess** — confirm licensing (**E5 / E5 Compliance / Information Protection & Governance** for

EDM), the EDM/authoring roles, the **EDM_DataUploaders** authorization, the in-scope data source, and data-owner/privacy sign-off.

- **Design** — define the schema (fields, primary vs supporting, configurable match), the SIT logic

(primary element + supporting elements + proximity + confidence), the source-file export, the DLP pilot policy, and **name every object** — with the data owners.

- **Configure** — create the schema and SIT, then hash-and-upload the data with the Upload Agent, and

confirm the data store indexed.

- **Pilot** — build the DLP policy in **simulation/test**, scoped to a pilot ring, and confirm it

would match on real values without blocking anyone.

- **Validate** — run the pass/fail gates in `05-testing.md` (exact match hits, near-miss correctly

ignored, supporting-element/proximity behavior, false-positive rate).

- **Expand** — enforce to the pilot ring, widen deliberately, and stand up the **refresh schedule**.

What's in the box

- This overview, a **Requirements** guide, an ordered **Implementation** guide, **Testing** (pass/fail

gates), **Troubleshooting**, **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.

- **Five PowerShell tools** (Security & Compliance PowerShell / Graph, wrapping the EDM Upload Agent

where relevant): an EDM **readiness/discovery** assessment, a **schema/rule-package** helper (`-WhatIf` first), a **validation** checker (test values against the SIT), a **reporting** rollup (schemas, SITs, data-store/refresh status, referencing policies), and a **scope/health** helper — read-only or `-WhatIf` by default; **no credentials, salts, or sensitive data in code**.

> ***Sources:*** *Microsoft Learn — *Learn about exact data match based sensitive information types*, *Get started with exact data match based sensitive information types* (schema, EDM Upload Agent, hashing/upload, refresh), *Create custom sensitive information types with EDM*, and the DLP simulation-mode documentation. EDM is* ***licensing-dependent*** *(Microsoft 365* ***E5 / E5 Compliance / Information Protection & Governance****) and the Purview portal, schema limits, and Upload Agent switches* ***evolve*** *—* ***verify capability, limits, and portal paths against your licensing/tenant*** *rather than assuming.*

2. Requirements — Exact Data Match (EDM) in Microsoft Purview

These are the **solution-specific** prerequisites for configuring EDM, **on top of** the six category prerequisites in `global-requirements-purview.md` (tenant/licensing, least-privilege roles, the S&C-PowerShell/Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, and a pilot scope + change control). Verify each item below — with its **Description / Why / Verify / Implement** — before starting the Implementation Guide.

> ***Boundary reminder.*** *EDM is* ***licensing-dependent*** *and the portal/limits* ***evolve*** *— treat every version number and portal path here as* ***"verify in your tenant."*** **Which* records are lawful to load, their* ***minimisation and retention****, are* ***privacy/compliance/legal determinations*** *(GR §5) — this solution provides the technical method and evidence, not the determination.*

R1 — EDM-capable licensing (E5 / E5 Compliance)

• **Description:** EDM-based classification requires **Microsoft 365 E5**, the **E5 Compliance**

add-on, or the **Information Protection & Governance** license on the tenant (and on users where the consuming capability — e.g. auto-labeling — is per-user licensed).

• **Why required:** EDM schemas and EDM SITs are not available without the qualifying plan; the

Exact data match node will be absent or read-only otherwise.

• **How to verify:** M365 admin center → **Billing** → **Licenses** confirms the SKU/add-on; in the

Purview portal, **Data classification** → **Classifiers** → **Exact data match (EDM)** is present and lets you create a schema.

• **How to implement if missing:** Assign/purchase the qualifying licensing before starting.

(Verify the current EDM licensing matrix on Microsoft Learn — plan names change.)

R2 — EDM authoring roles (least privilege)

• **Description:** An admin account holding a role that can **create/manage EDM schemas and SITs** —

typically membership in **Compliance Administrator** or **Compliance Data Administrator** (the **Organization Management** / **Security Administrator** role groups also qualify), assigned in the Purview portal. Global Administrator is **not** required and should not be used for routine work.

• **Why required:** Creating the schema, the rule package, and the SIT is gated on these role groups (GR §2 — least privilege).

• **How to verify:** Purview portal → **Settings** → **Roles & scopes** → **Permissions** (or the *Microsoft Purview* role groups) confirms the account's membership; the account can open the EDM wizard and see **Create EDM schema**.

• **How to implement if missing:** Add the account to the least-privilege qualifying role group under change control.

R3 — EDM_DataUploaders authorization (upload account)

• **Description:** The account that will run `EdmUploadAgent.exe` to hash and upload the data must be a member of the `EDM_DataUploaders` security group in **Microsoft Entra ID** (the group name is fixed and expected by the service).

• **Why required:** The Upload Agent authenticates as this account; upload is **rejected** if the account is not in `EDM_DataUploaders`.

• **How to verify:** In Entra ID / Graph, confirm the group `EDM_DataUploaders` exists and the upload account is a member; `EdmUploadAgent.exe /Authorize` completes sign-in without an authorization error. `PURVEDM-Discovery.ps1` reports the group and membership.

• **How to implement if missing:** Create the `EDM_DataUploaders` security group in Entra ID and add the upload account, then re-authorize. **(Verify the exact required group name / any tenant-specific variation on Microsoft Learn.)**

R4 — The EDM Upload Agent installed for the correct cloud

• **Description:** The **EDM Upload Agent** (`EdmUploadAgent.exe`) installed on a domain-joined, access-controlled Windows host **from the download that matches your cloud instance** — **Commercial**, **GCC**, **GCC High**, or **DoD** each have a **different** Upload Agent build and endpoint.

• **Why required:** The Agent is the **only** way to hash and upload; a mismatched-cloud build will fail to authenticate/upload. (Government tenants **must** use their cloud's build — see the Gov note below.)

• **How to verify:** `EdmUploadAgent.exe /?` runs and reports its version; `/Authorize` succeeds against your cloud.

• **How to implement if missing:** Download and install the Upload Agent build for your cloud from the current Microsoft Learn "Get started with EDM" page. **(Download links and switches evolve — verify the current page and run `EdmUploadAgent.exe /?`.)**

> ***Government (GCC High / DoD) note.*** *Sovereign clouds use* ***distinct*** *Upload Agent downloads, service endpoints, and sometimes portal URLs (e.g.* `purview.microsoft.us`*). Confirm the correct build and endpoints for your cloud before hashing — a Commercial-cloud agent will not work in GCC High/DoD.*

R5 — The sensitive-data source and schema design (with the data owner)

• **Description:** A defined **source records set** you can export to `.csv`/`.tsv`, and an agreed **schema** design: the fields (columns), which field is the **primary/searchable element**, which are **supporting elements**, and the per-field **configurable-match** options (ignore case / ignore delimiters). Header row field names must match the schema exactly.

• **Why required:** The schema and the SIT logic both key off this design; a wrong primary element or mismatched header breaks matching. Field-count and row/size limits apply.

• **How to verify:** Confirm with the **data owner** the columns, the primary element, the supporting elements, and that an export in the required format is producible within the documented limits. `PURVEDM-Schema.ps1 -WhatIf` validates a candidate schema definition.

• **How to implement if missing:** Work with the data owner to define the schema and produce a representative export. **(Verify current field-count, row-count, and file-size limits on Microsoft Learn — they have increased over time.)**

R6 — A secure host and secret handling for hashing (salt + plaintext)

• **Description:** An **access-controlled host** to run the Agent on, a plan for the **salt** (supply a secret salt or let the Agent generate one — either way **record it securely and reuse it**), and a procedure to **securely delete the plaintext export** after hashing.

• **Why required:** The export is live sensitive data and the salt is a secret that makes hashes non-reversible and enables future matching/refresh. Mishandling either is a data-exposure risk.

• **How to verify:** The host has restricted access; a secret store holds the salt; a secure-deletion step is defined. **No salt or plaintext appears in any script, log, or repo** (the provided scripts never take a salt or read source data).

• **How to implement if missing:** Provision the host, choose the salt-handling approach, and define secure deletion before exporting any data.

R7 — In-scope workloads and a consuming policy design

• **Description:** The **workloads** where the EDM SIT will be enforced (Exchange, SharePoint, OneDrive, Teams, Endpoint — GR §4) and the **consuming policy** design: at minimum a **DLP policy** (this solution's default), optionally auto-labeling or retention that references the EDM SIT.

• **Why required:** An EDM SIT does nothing until a policy uses it; the workloads and action are what produce protection.

• **How to verify:** Confirm the target workloads and the intended DLP policy/rule (name, conditions, action) with compliance.

• **How to implement if missing:** Design the DLP policy and rule to reference the EDM SIT, to be built in **simulation** in the Implementation Guide.

R8 — Pilot scope, change control, and the refresh plan

• **Description:** A named **pilot ring** (`SG-EDM-Pilot`) to validate on, a **change-control** path for simulate→enforce, and an owner/cadence for the **data refresh** (re-hash + re-upload as the source data changes; the data store must not go stale).

• **Why required:** DLP is high-impact (pilot in simulation first — GR §6), and EDM matching quality **decays** if the hashed data isn't refreshed.

• **How to verify:** A named pilot ring exists, a change window/approver is identified, and a refresh owner + cadence (and, ideally, a scheduled re-upload task) are agreed.

• **How to implement if missing:** Define the pilot ring, the change-control path, and the refresh schedule before enforcing. `PURVEDM-Set-Scope.ps1` helps manage the pilot ring; the Best Practices guide covers automating refresh.

Requirements checklist

#	Requirement	Verify with
R1	EDM-capable licensing (E5 / E5 Compliance)	Admin center licenses; EDM node present
R2	EDM authoring role (least privilege)	Purview Roles & scopes
R3	`EDM_DataUploaders` membership (upload account)	Entra group + `/Authorize`; `PURVEDM-Discovery.ps1`
R4	EDM Upload Agent for the correct cloud	`EdmUploadAgent.exe /?` + `/Authorize`
R5	Source data + schema design (with data owner)	Data-owner sign-off; `PURVEDM-Schema.ps1 -WhatIf`
R6	Secure host + salt/plaintext handling	Host ACLs; secret store; secure-delete step
R7	In-scope workloads + consuming DLP policy design	Compliance-agreed policy/rule/action
R8	Pilot ring, change control, refresh plan	Named ring/approver/refresh owner + cadence

> *The six **category** prerequisites in `global-requirements-purview.md` still apply in full — this list is additive.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation — Exact Data Match (EDM) in Microsoft Purview

This is the ordered, do-this-then-that guide. It follows the standard model (**Assess** → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand**) and — because EDM loads a hashed copy of **real** sensitive records — it confirms **licensing, roles, upload authorization, and data-owner/privacy sign-off FIRST**, then hashes/uploads with the plaintext kept local, then builds enforcement in **simulation** before anything blocks. Every object is named verbatim so the scripts, testing, and troubleshooting line up. Replace the `contoso.com` / `CustomerRecords` sample names with your own.

> ***Named objects used throughout (rename to your own):*** *- **EDM schema / data store name:*** `CustomerRecords` *- **Fields:*** `AccountNumber` *(**primary / searchable**, ignore delimiters),* `LastName` *(supporting),* `SSN` *(supporting, ignore delimiters),* `Email` *(supporting) - **EDM sensitive information type:*** `EDM - Customer Account Match` *- **Entra security group (upload auth):*** `EDM_DataUploaders` **(fixed name)* - **Source export:*** `C:\EDM\CustomerRecords.csv` *— **Hash output folder:*** `C:\EDM\Hash` *- **DLP pilot policy:*** `DLP-EDM-CustomerRecords-Sim` *(simulation), rule* `EDM-CustomerRecords-Block`*, scoped to pilot ring* `SG-EDM-Pilot`
>
> ***Portal note.*** *The EDM **schema + SIT** are authored in **Microsoft Purview portal** (*`purview.microsoft.com` *→ **Data classification** → **Classifiers** → **Exact data match (EDM)**), or via **PowerShell/XML**. The **hash + upload** step is **always*** `EdmUploadAgent.exe` *— there is no plaintext upload in the portal. The **DLP policy** is authored in **Purview** → **Data Loss Prevention**. Where a path, limit, or Agent switch may differ by tenant/cloud/version, **verify against Microsoft Learn and your tenant.***
>
> ***Hard gate before you enforce.*** *Do **not** move the DLP policy from simulation to enforce until R8 (pilot validated) **and** the data-owner/privacy sign-off on loading these records (R5/R6, GR §5) are satisfied **in writing**.*

Phase 0 — Assess (confirm you can build safely)

1. **Confirm licensing (R1).** E5 / E5 Compliance / Information Protection & Governance present; the **Exact data match (EDM)** node is visible and lets you start a schema.

1. **Confirm authoring role (R2)** and the `EDM_DataUploaders` group + upload-account membership (R3). Run `PURVEDM-Discovery.ps1` (read-only) — it reports EDM licensing signals, existing schemas/SITs, and the `EDM_DataUploaders` group/membership via Graph.

• **Gate 0:** licensing present, authoring role held, `EDM_DataUploaders` exists with the upload account in it → continue. Otherwise fix the missing prerequisite first.

1. **Confirm the Upload Agent for your cloud (R4).** `EdmUploadAgent.exe /?` runs; `/Authorize` succeeds. **Government tenants: confirm the GCC/GCC High/DoD build and endpoints.**

1. **Confirm the toolchain** (GR §3): `Connect-IPPSSession` and `Connect-MgGraph` succeed with an MFA-capable admin.

Phase 1 — Design (agree it with the data owner, then name everything)

Decide and **record with the data owner / privacy / compliance**:

• **The schema (R5).** The fields and their roles, e.g.:

Field	Role	Configurable match	Notes
`AccountNumber`	**Primary / searchable**	**Ignore delimiters**	the exact value a document must contain
`LastName`	Supporting	Ignore case	corroborates the match
`SSN`	Supporting	Ignore delimiters	corroborates; never the sole element
`Email`	Supporting	Ignore case	corroborates the match

• **At least one** field must be **searchable/primary**. Choose a field with **high cardinality and a stable format **(an account/record ID) as primary — not a low-cardinality field like a last name.** (Verify current max field count / searchable-field count in Microsoft Learn.)**

• **The SIT logic.** *Match when a document contains an exact `AccountNumber` value **and** at least **one supporting element** (`LastName` / `SSN` / `Email`) within a **proximity** window (e.g. the default character window), at **high confidence**.* Define a second, higher-confidence variant (primary + **two** supporting elements) if you want a stricter tier.

• **The source export.** How `C:\EDM\CustomerRecords.csv` will be produced (header row = `AccountNumber,LastName,SSN,Email`), within the documented row/size limits, on the secure host (R6).

• **The DLP pilot policy** and verbatim names (Phase 5):

• Policy `DLP-EDM-CustomerRecords-Sim` — scoped to `SG-EDM-Pilot`, **simulation/test** first.

• Rule `EDM-CustomerRecords-Block` — condition = the EDM SIT `EDM - Customer Account Match`; action (once enforced) = **block / block-with-override** per compliance.

Phase 2 — Create the EDM schema and SIT

Portal path (recommended): Purview → **Data classification** → **Classifiers** → **Exact data match (EDM)** → **EDM schemas** → **Create EDM schema**:

1. **Name** the schema `CustomerRecords` and add a description.

2. **Add the fields** `AccountNumber`, `LastName`, SSN, `Email`. Mark **`AccountNumber` as

searchable (primary)**; set** configurable match** per field (ignore case / ignore delimiters) as designed in Phase 1.

1. **Save** the schema (data store `CustomerRecords`).

2. **Create the EDM sensitive information type** `EDM - Customer Account Match` referencing the schema:

set the **primary element** (`AccountNumber`), the **supporting elements** and their **proximity**, and the **confidence level(s)** per the Phase 1 logic.

PowerShell/XML path (repeatable/source-controlled): author the schema XML and rule package, then:

```
New-DlpEdmSchema -FileData ([System.IO.File]::ReadAllBytes('CustomerRecords-schema.xml'))
New-DlpSensitiveInformationTypeRulePackage -FileData ([System.IO.File]::ReadAllBytes('CustomerRecords-rule
```

`PURVEDM-Schema.ps1 -WhatIf` validates a schema XML and prints the intended `New-DlpEdmSchema` call; `-Commit` creates/updates it.

• **Pass/fail:** `Get-DlpEdmSchema` lists `CustomerRecords`; `Get-DlpSensitiveInformationType` lists

`EDM - Customer Account Match`. **Rollback:** remove the SIT then the schema (`Remove-DlpSensitiveInformationType` / `Remove-DlpEdmSchema`) — nothing matches until data is uploaded anyway.

*The schema must exist **before** you can upload data to its data store.*

Phase 3 — Export, hash, and upload the sensitive data (plaintext stays local)

On the **secure host** (R6), signed in as the `EDM_DataUploaders` upload account:

1. **Export** the records to `C:\EDM\CustomerRecords.csv` — first row exactly

`AccountNumber,LastName,SSN,Email`, one record per row, within the documented limits.

1. **Authorize the Agent** (one-time / as needed):

```
EdmUploadAgent.exe /Authorize
```

1. **(Optional) Test the hash locally first** — hash **without** uploading to validate the format and

your salt handling:

```
EdmUploadAgent.exe /CreateHash /DataStoreName CustomerRecords /Format csv `
    /SourceFile C:\EDM\CustomerRecords.csv /HashLocation C:\EDM\Hash /Salt <your-secret-salt>
```

*(If you omit `/Salt`, the Agent generates one — **record it securely and reuse it** for every refresh.)*

1. **Hash and upload** the data to the `CustomerRecords` data store:

```
EdmUploadAgent.exe /UploadData /DataStoreName CustomerRecords /DataFile C:\EDM\CustomerRecords.csv `
    /HashLocation C:\EDM\Hash /Format csv /AllowedBadLinesPercentage 5
```

1. **Confirm the upload/index status:**

```
EdmUploadAgent.exe /GetSession /DataStoreName CustomerRecords
EdmUploadAgent.exe /GetDataStore /DataStoreName CustomerRecords
```

1. **Securely delete** the plaintext `C:\EDM\CustomerRecords.csv` once the upload is confirmed (R6).

> ***Allow time to index.*** *After a successful upload the data store needs* ***time to index*** *before matching works reliably (Microsoft documents this can take up to roughly an hour, sometimes longer for large data) —* ***do not judge matching before indexing completes.*** `PURVEDM-Report.ps1` *(and the Agent's* `/GetDataStore`*) surface the data-store's row count / last-upload signal.*

• **Pass/fail:** `/GetSession` reports the upload **completed**; `/GetDataStore` shows the expected

row count and a recent upload timestamp. **Rollback:** re-upload a corrected file (upload replaces the data store's data); the SIT simply matches against whatever is currently uploaded.

> ***Switches vary by Agent version/cloud.*** *Confirm the exact switches with* `EdmUploadAgent.exe /?` *and the current Microsoft Learn "Get started with EDM" page —* ***do not*** *assume a switch that your build doesn't list.*

Phase 4 — Validate the SIT matches your data (before any policy)

Once indexing has had time to complete, **test the SIT with non-production sample values you are authorised to use** (a handful of test records, or values you own):

1. **Portal Test:** Purview → **Data classification** → **Classifiers** → **Sensitive info types** → open

`EDM - Customer Account Match` → **Test** → upload a document containing a **real primary value + a supporting value** → confirm it matches; upload a document with a **primary value that is NOT in your data** → confirm it does **not** match.

1. **PowerShell:** `PURVEDM-Validate.ps1` uses `Test-DataClassification` to run sample text against the

EDM SIT and report whether it matched (exact-value hit + supporting element/proximity) — read-only.

• **Pass/fail:** an in-data primary value **with** a supporting element matches; a **not-in-data**

value does **not** match; a primary value **without** any supporting element in proximity does **not** match (confirms EDM's exactness). If nothing matches, re-check indexing time, the header row, and the primary-element choice (see `06-troubleshooting.md`).

Phase 5 — Build the DLP enforcement in SIMULATION/TEST first

In **Purview** → **Data Loss Prevention** → **Policies** → **Create policy** (custom), create `DLP-EDM-CustomerRecords-Sim`, scoped to `SG-EDM-Pilot`, on the in-scope workloads (R7), with the policy **mode set to simulation / test** (*Test it out first*). Add the rule:

• `EDM-CustomerRecords-Block`

• **Condition:** *Content contains* → **Sensitive info types** → `EDM - Customer Account Match`

(optionally combine with a sensitivity-label condition to scope to labeled content).

• **Action (defined now, enforced later): Block** / **Block with override** per compliance, with

a policy tip.

`PURVEDM-Schema.ps1`/`PURVEDM-Set-Scope.ps1` help stage the policy scope; the DLP simulation itself is set in the portal (or with the DLP cmdlets in a Test* mode — never Enable here).

- **Pass/fail:** the policy shows **On (test/simulation)** and **healthy**; simulation begins reporting matches once pilot content flows. **Rollback:** set the policy **mode = simulation/Disable** — it never blocks users while in test.

Phase 6 — Pilot, validate, then expand (enforce, then widen)

1. **Validate** — work the gates in `05-testing.md`: exact-match hits, near-miss/not-in-data correctly ignored, supporting-element/proximity behavior, and a measured **false-positive rate** (EDM's whole point is that this is near-zero). Reconcile with **Activity explorer** / DLP alerts.
1. **Enforce to the pilot ring** — only after the gates pass **and** the R5/R6 data-owner/privacy sign-off is in writing: change `DLP-EDM-CustomerRecords-Sim` **mode from simulation to enforce** (turn on the action/policy tips). Watch the first cohort closely.
1. **Widen scope deliberately** — group-by-group via `PURVEDM-Set-Scope.ps1` (`-WhatIf` first), re-running the gates after each expansion.
1. **Stand up the refresh (R8)** — schedule a re-hash + re-upload (`/UploadData`) of the source data on the agreed cadence so the data store never goes stale (see Best Practices for automating this on the secure host).

Rollback summary

Change	Rollback
Schema/SIT wrong	`Remove-DlpSensitiveInformationType` then `Remove-DlpEdmSchema` (or fix + re-create); nothing matches until data is uploaded.
Bad / stale uploaded data	Re-run `EdmUploadAgent.exe /UploadData` with a corrected export — the upload replaces the data store's data.
SIT not matching	Re-check indexing time, header row, primary-element choice, configurable-match options; re-test with `PURVEDM-Validate.ps1`.
DLP rule too aggressive / mis-scoped	Set `DLP-EDM-CustomerRecords-Sim` **mode = simulation** (instant, no data change); adjust conditions/scope and re-pilot.
Scope too wide	Remove members from `SG-EDM-Pilot` (`PURVEDM-Set-Scope.ps1`); the policy follows the group.

> ***Never** put the **salt**, the **plaintext export**, or any real record values into a script, log, or repository. The provided scripts never read source data or take a salt — the hashing/upload is done only by the Upload*

Agent on the secure host.

5. Testing & Validation — Exact Data Match (EDM) in Microsoft Purview

Validate the EDM implementation against explicit **pass/fail gates**. The three headline gates are: **(A) it matches your real records exactly**, **(B) it does NOT match look-alikes / not-in-data values** (the whole point of EDM), and **(C) the SIT enforces correctly in a policy**. Run every test with **values you are authorised to use for testing** — a small set of test records you control, or sanctioned sample values — and record the evidence. **Never** paste real customer/employee records into ad-hoc test documents outside the sanctioned set.

> *All tests assume the schema/SIT from the Implementation Guide, the data **hashed & uploaded** and **fully indexed** (allow the documented indexing time before judging results), and the DLP policy proven in **simulation** before enforcement.*

Test A — Exact match on real data (it detects what it should)

#	Test	Steps	Pass condition
A1	Primary + supporting match	Put an **in-data** `AccountNumber` value **and** a matching supporting value (`LastName`/`SSN`/`Email`) in a test document; run the portal **Test** on the SIT (or `PURVEDM-Validate.ps1`).	The SIT **matches** at the expected confidence.
A2	Configurable-match (delimiters)	Use an in-data value with **different delimiters** than stored (e.g. `123456789` vs stored `123-45-6789`) on a field flagged *ignore delimiters*.	Still **matches** — confirms the configurable-match option works.
A3	Configurable-match (case)	Vary the **case** of a supporting value on a field flagged *ignore case*.	Still **matches**.
A4	Second-tier (stricter) confidence	If a higher-confidence variant (primary + **two** supporting elements) was defined, test a document meeting it.	Matches at the **higher** confidence tier.

Gate A (pass): in-data values (with a supporting element in proximity) match, and the configurable match options behave as designed.

Test B — Exactness (it does NOT match look-alikes) — the defining EDM gate

#	Test	Steps	Pass condition
B1	Not-in-data value	Put a **correctly formatted but NOT-in-your-data** primary value (e.g. a random valid-looking account number) + a supporting value in a document.	**No match** — this is the core EDM advantage over pattern SITs.
B2	Primary with no supporting element	Put an **in-data** primary value **alone**, with no supporting element in proximity.	**No match** (or only the primary-only tier if you deliberately defined one) — confirms supporting-element/proximity logic.
B3	Supporting element only	Put a supporting value (e.g. a common `LastName`) **without** any in-data primary value.	**No match** — a supporting element alone never triggers.
B4	Pattern-SIT comparison (optional)	Run the same B1 document past the equivalent **built-in pattern SIT**.	The pattern SIT **would** match (false positive) while EDM does **not** — demonstrates the false-positive reduction.

Gate B (pass): not-in-data and unsupported values are correctly **not** matched — EDM is exact.

Test C — Policy enforcement (the control is operable)

#	Test	Steps	Pass condition
C1	Simulation match	With `DLP-EDM-CustomerRecords-Sim` in **simulation**, move a document with an in-data match through a covered workload (e.g. email to external, external share).	Simulation reports the rule **would** act; a simulation match/alert is recorded — **no user impact**.

#	Test	Steps	Pass condition
C2	Enforced action (pilot ring)	After enforcing to `SG-EDM-Pilot`, repeat C1 as a pilot-ring user.	The action (block / block-with-override) fires; the user sees the policy tip; a DLP **alert** is raised.
C3	Alert + evidence trail	Locate the match in **Purview** → **DLP** → **Alerts** and **Data classification** → **Activity explorer**.	Alert present with the correct policy/severity; full evidence trail (user, location, SIT, action) in Activity explorer.
C4	Scope containment	As a user **outside** `SG-EDM-Pilot`, repeat C1 during pilot.	**Not** enforced — confirms scope containment.

Gate C (pass): simulation reports would-act with no impact; enforced action fires for the ring and is alerted/trailed; out-of-scope users are unaffected.

Test D — Data store health & refresh (matching stays accurate)

#	Test	Steps	Pass condition
D1	Upload/index confirmed	`EdmUploadAgent.exe /GetDataStore /DataStoreName CustomerRecords` (and `PURVEDM-Report.ps1`).	Expected **row count** and a recent **upload timestamp**; session **completed**.
D2	Refresh applied	After a refresh (`/UploadData` of updated data), add a **newly-added** record's value to a test doc and re-test (A1).	The **new** value now matches — confirms refresh took effect.
D3	Removed record no longer matches	After a refresh that **removed** a record, test that record's value.	**No longer matches** — confirms removals propagate.

Gate D (pass): the data store reports healthy with the expected rows; adds/removes propagate after refresh.

Simulation-first & rollback validation

#	Test	Pass condition
S1	Nothing enforced outside the ring	Users outside `SG-EDM-Pilot` experience no blocking during pilot (same as C4).
S2	Instant rollback	Setting `DLP-EDM-CustomerRecords-Sim` back to **simulation** immediately removes enforcement with no data change (a previously-blocked action now only simulates).
S3	No secrets in tooling	Confirm no script, log, or repo contains the **salt**, the **plaintext export**, or real record values (the provided scripts never read source data or accept a salt).

Evidence to capture

- Portal **Test** results / `PURVEDM-Validate.ps1` output for A1–A4 and B1–B3.
- Simulation match report and the enforced **DLP alert** + **Activity explorer** trail (C1–C3).
- `PURVEDM-Report.ps1` / `/GetDataStore` output showing data-store health (D1) and refresh (D2–D3).
- The measured **false-positive rate** (expected near-zero) versus the equivalent pattern SIT (B4).

> ***Boundary:*** *the tests prove the* ***technical control and its evidence****. *Which* records are in scope, the* ***enforced action****, and the* ***retention of the loaded data*** *are compliance/privacy/legal determinations, captured alongside the evidence.*

6. Troubleshooting — Exact Data Match (EDM) in Microsoft Purview

Common EDM failure modes and how to resolve them. Work top-down; most "it isn't matching" cases are **indexing time**, a **header/primary-element mismatch**, or a **stale/failed upload** — not a broken SIT. **Verify licensing/tenant, limits, and Upload Agent switches** as you go — they evolve.

The EDM node / schema creation is missing or greyed out

- **Most likely licensing (R1).** EDM requires **E5 / E5 Compliance / Information Protection &

Governance**. Confirm the SKU in the admin center; the *Exact data match (EDM)* node under** Data classification → Classifiers** should then let you create a schema.

- **Or role (R2).** You need an **EDM authoring role** (Compliance Administrator / Compliance Data

Administrator, or Organization Management / Security Administrator). Confirm membership in Purview → **Roles & scopes**. `PURVEDM-Discovery.ps1` surfaces both signals.

Upload fails with an authorization / access error

- **Account not in** `EDM_DataUploaders` **(R3).** The Upload Agent account **must** be a member of the `EDM_DataUploaders` security group in Entra ID. Confirm the group exists and the account is a member (`PURVEDM-Discovery.ps1`), then re-run `EdmUploadAgent.exe /Authorize`.
- **Wrong-cloud Agent (R4).** A **Commercial** Upload Agent will not authenticate against **GCC / GCC High / DoD **(and vice-versa). Download and use the** build for your cloud**, with your cloud's endpoints.
- **Stale authorization.** Re-run `/Authorize` and complete modern-auth/MFA sign-in as the upload account.

`EdmUploadAgent.exe` errors on the data file

- **Header row mismatch.** The **first row** of the `.csv`/`.tsv` must be the **field names**, matching the **schema** exactly (names and count). A mismatch is the most common upload/hashing error.
- **Wrong format / separator.** Match `/Format csv` (or `tsv`) to the actual file; if your data uses a non-standard separator, set the column-separator switch. Confirm the file isn't quoted/encoded in a way the Agent can't parse.
- **Bad lines.** If some rows are malformed, `/AllowedBadLinesPercentage <n>` lets the upload proceed while skipping them — but investigate *why* rows are bad rather than raising the threshold blindly.
- **Switch not recognised.** Switches vary by Agent version/cloud — run `EdmUploadAgent.exe /?` and confirm against the current Microsoft Learn page; don't assume a switch your build doesn't list.

The SIT doesn't match values you KNOW are in the data

- **Give it time to index.** After a successful upload the data store must **index** before matching is reliable — Microsoft documents this can take up to roughly an hour (longer for large data). Confirm the upload **completed** (`/GetSession`) and the data store shows the expected **row count** (`/GetDataStore` or `PURVEDM-Report.ps1`) before concluding it's broken.
- **Wrong primary/searchable element.** A document only becomes a candidate if it contains an **exact match to a searchable (primary) field**. If your primary element is a low-value field (or the wrong one), matching won't trigger — review the schema (Phase 1) and re-create with a high-cardinality primary element.
- **Configurable match not set.** If documents format the value differently than stored (e.g. `123456789` vs `123-45-6789`), the field needs **ignore delimiters** / **ignore case**. Set the option and re-test.

- **No supporting element in proximity.** If the SIT requires a supporting element and your test document has only the primary value, it correctly won't match — add a supporting value in proximity (or test the primary-only tier if you defined one). Use `PURVEDM-Validate.ps1` to isolate this.

The SIT matches values that are NOT in the data (over-matching)

- **The rule relies on the primary element alone.** Require **one or more supporting elements in proximity** so a lone look-alike value can't trigger. Tighten the SIT and re-validate (Test B).
- **You're actually hitting a different (pattern) SIT.** A built-in pattern SIT for the same data category may be matching, not your EDM SIT. Check *which* SIT fired in the match evidence; scope the policy to the **EDM SIT** specifically.

Newly-added records don't match / removed records still match

- **The data store is stale (R8).** EDM matches the **last uploaded** hashes. New records won't match and removed records keep matching until you **re-hash + re-upload** (`EdmUploadAgent.exe /UploadData`). Confirm the last-upload timestamp (`PURVEDM-Report.ps1`) and stand up an automated **refresh schedule** (Best Practices).
- **Refresh failed silently.** A scheduled refresh that errors is invisible until matching degrades — alert on refresh failures and verify each run (Test D2/D3).

The DLP policy isn't acting on EDM matches

- **Still in simulation.** By design the policy is built in **simulation/test** first — it reports would-act without enforcing. Confirm the mode; enforce only after the gates pass and sign-off is in writing (Phase 6).
- **Scope / workload mismatch (R7).** Confirm the policy covers the workload you're testing and includes your pilot user/group (`PURVEDM-Set-Scope.ps1`). A user outside `SG-EDM-Pilot` is correctly unaffected during pilot.
- **Condition not referencing the EDM SIT.** Confirm the rule's *Content contains → Sensitive info types* lists `EDM - Customer Account Match` (`PURVEDM-Report.ps1` shows which policies reference each EDM SIT).

`Test-DataClassification` / validation script returns nothing

- **SIT name mismatch.** Pass the **exact** SIT name; confirm with `Get-DlpSensitiveInformationType`.
- **Not yet indexed.** Same as above — a freshly uploaded data store needs indexing time before `Test-DataClassification` will return EDM hits.
- **Connection.** Ensure `Connect-IPPSSession` succeeded and `Test-DataClassification` is available in

your session.

Government / sovereign-cloud specifics

• Use the **GCC High / DoD** Upload Agent, endpoints, and portal (`purview.microsoft.us` where applicable). A Commercial-cloud build **will not** work. Verify the correct downloads/endpoints for your cloud before hashing.

When to escalate vs. defer

• **Technical** (upload errors, indexing, schema/SIT logic, policy wiring) → work the steps above; use `PURVEDM-Discovery.ps1` / `PURVEDM-Validate.ps1` / `PURVEDM-Report.ps1` to isolate.

• **Determination** (*which* records may be loaded, the enforced action, minimisation/retention of the loaded data) → **defer to compliance/privacy/legal** (GR §5). This solution provides the technical method and the evidence, not the determination.

7. Best Practices — Exact Data Match (EDM) in Microsoft Purview

Field-tested practices for running EDM so it stays **exact, current, and privacy-preserving**. These complement the Implementation and Testing guides.

Protect the plaintext and the salt — non-negotiable

• **The plaintext export never leaves the secure host.** Produce `C:\EDM\*.csv` under change control on an **access-controlled** machine, hash it there, and **securely delete** the plaintext once the upload is confirmed. Only the hashes ever go to the service.

• **Treat the salt as a secret and reuse it.** Whether you supply the salt or let the Agent generate one, **record it in a secret store** and use the **same salt** for every refresh — it is part of what makes hashes non-reversible and keeps matching consistent.

• **Never** put the salt, the plaintext, or any real record values into a script, log, ticket, or repository. The provided scripts deliberately **never read source data or take a salt**.

Design the schema for exactness and performance

• **Choose a high-cardinality, stable-format field as the primary/searchable element** — an account or record ID, **not** a name or a low-cardinality field. The primary element is the lookup key; a poor choice bloats the index and weakens matching.

• **Always require supporting elements.** A primary value alone is a weak signal; requiring one or two

supporting elements in proximity is what drives false positives toward zero. Define a **stricter, higher-confidence tier** (primary + two supporting) for your most sensitive actions.

- **Use configurable match deliberately.** *Ignore delimiters* for values people format

inconsistently (SSNs, account numbers); *ignore case* for names/emails. Match the option to how the data actually appears in documents.

- **Keep the schema lean.** Only include the fields you actually match on or corroborate with; respect

the current field-count limit. **(Verify current field / row / file-size limits on Microsoft Learn — they have increased over time.)**

Refresh the data — a stale EDM data store silently fails

- **EDM matching decays as the source data drifts.** New records won't match and removed records keep

matching until you refresh. Schedule a **re-hash + re-upload** (`EdmUploadAgent.exe /UploadData`) on a cadence that matches how fast the data changes (often nightly or weekly).

- **Automate the refresh on the secure host** — a scheduled task that regenerates the export, hashes

with the **stored salt**, uploads, and securely deletes the plaintext. Alert on failures; a silently failed refresh is invisible until matching quality drops.

- **Verify each refresh took effect** (Test D2/D3): a newly-added value now matches; a removed value no

longer does.

Simulate before you enforce — always

- **Never** switch the consuming DLP policy straight to enforce. Run `DLP-EDM-CustomerRecords-Sim` in

simulation first, confirm it matches real data and ignores look-alikes, then enforce to the **pilot ring**, then expand.

- Treat **simulation mode as your permanent safety switch** — any enforced policy returns to simulation

in one setting, instantly removing enforcement with no data change.

Use EDM where pattern SITs over-fire — not everywhere

- EDM's payoff is highest for categories that **pattern SITs match too loosely** (account numbers,

member IDs, customer PII) where you have an authoritative source table. For genuinely pattern-defined data with good built-in detectors, a built-in/custom SIT may be simpler.

- **Combine EDM with sensitivity labels** in DLP rules when you want protection to key on **labeled +

exactly-matched** content — the strongest, most precise tier.

- **Reuse one EDM data store across policies** (DLP, auto-labeling, retention, Communication

Compliance) rather than maintaining several near-duplicate sources.

Operations & governance

• **Give the data store an owner and a refresh SLA** (R8). Match volume without a current data store is not a reliable control.

• **Version and document** each schema (fields, primary/supporting, configurable match, confidence, owner) and each refresh, so changes are traceable.

• **Least privilege and separation of duties.** Keep **authoring** (schema/SIT), **uploading** (`EDM_DataUploaders`), and **alert triage** as distinct roles where possible.

• **Confirm the correct cloud build** for Government tenants (GCC/GCC High/DoD) — a Commercial Upload Agent will not work in a sovereign cloud.

Stay inside the boundary

• **Verify licensing/tenant, limits, and Agent switches** before relying on any capability — plans, portal paths, schema limits, and `EdmUploadAgent.exe` switches evolve; run `EdmUploadAgent.exe /?` and check Microsoft Learn rather than assuming.

• EDM is **one control**, not a guarantee — it matches content it can read and classify; combine with labels, encryption, and insider-risk where the risk warrants.

• **Which records are lawful to load, their minimisation, and their retention** are **privacy/compliance/legal determinations** — you provide the technical method and the evidence; they own the decision.

8. Deliverables Summary — Exact Data Match (EDM) in Microsoft Purview

What you receive in this solution, and what "done" looks like.

Documentation

Document	What it gives you
Solution Overview	The EDM model — the four parts (schema, source table, hashed/uploaded data, EDM SIT), how the plaintext stays local while only hashes are uploaded, and the Assess→Design→Configure→Pilot(simulate)→Validate→Expand journey; naming convention.

Document	What it gives you
Requirements Guide	Solution-specific prerequisites (EDM licensing, authoring roles, `EDM_DataUploaders` authorization, the correct-cloud Upload Agent, source data + schema design, secure host + salt/plaintext handling, consuming DLP policy, pilot ring + refresh plan) each with verify/implement steps, on top of the six category prerequisites.
Implementation Guide	The exact ordered phases: assess → design the schema with the data owner → create schema + SIT → **hash & upload with the EDM Upload Agent (plaintext stays local)** → validate the SIT → build DLP **in simulation** → enforce to a pilot ring → expand + refresh; each object named, gated, with a rollback.
Testing & Validation	Pass/fail gates for the headline outcomes — exact match on real data (A), non-match of look-alikes/not-in-data values (B), policy enforcement (C), and data-store health/refresh (D) — plus simulation-first and no-secrets checks, all with sanctioned test values.
Troubleshooting Guide	The common failure modes (SIT not matching, upload/authorization failures, indexing delays, stale data, wrong-cloud Agent, over/under-matching) and how to resolve them.
Best Practices	Plaintext/salt protection, schema design for exactness, the refresh discipline, simulate-first, where to use EDM vs pattern SITs, and operations/governance.
Executive Summary	One-page business framing for approvers.

PowerShell tools (5) — Security & Compliance PowerShell / Graph

All tools are **read-only or `-WhatIf` by default**; writes require an explicit `-Commit`. **No credentials, salts, or sensitive record data are stored in or read by any script** — hashing/upload is done only by `EdmUploadAgent.exe` on the secure host. They connect via `Connect-IPPSSession` and/or `Connect-MgGraph`.

#	Script	Role	Default safety
1	**PURVEDM-Discovery.ps1**	EDM readiness/discovery: EDM licensing signals, existing EDM schemas and EDM SITs, and the `EDM_DataUploaders` group + membership (via Graph).	Read-only

#	Script	Role	Default safety
2	**PURVEDM-Schema.ps1**	Schema/rule-package helper: validates a schema XML and prints the intended `New-DlpEdmSchema` call; `-Commit` creates/updates the schema. Can emit a starter schema-XML template.	`-WhatIf` (writes need `-Commit`)
3	**PURVEDM-Validate.ps1**	Validation checker: runs sample text/values through `Test-DataClassification` against the EDM SIT to confirm exact-match + supporting-element behavior.	Read-only
4	**PURVEDM-Report.ps1**	Reporting rollup: EDM schemas, their SITs, data-store row-count/last-upload signal (optionally via `EdmUploadAgent.exe /GetDataStore` when an exe path is provided), and DLP policies referencing the EDM SIT.	Read-only
5	**PURVEDM-Set-Scope.ps1**	Scope/health helper: reports and (`-Commit`) adjusts the DLP pilot policy's included/excluded users/groups (the pilot ring); surfaces refresh-scheduling guidance.	`-WhatIf` (writes need `-Commit`)

Definition of done

• The **EDM schema** and **EDM SIT** created (primary/searchable element + supporting elements + proximity + confidence), documented and versioned.

• The sensitive data **exported, hashed, and uploaded** with the EDM Upload Agent; **plaintext securely deleted**; the** salt **stored in a secret store; the data store** indexed and confirmed**.

• The SIT **validated** — matches in-data values, **ignores** not-in-data / unsupported values (Gates A & B).

• A **DLP policy** referencing the EDM SIT **proven in simulation**, then enforced to the pilot ring

and expanded (Gate C).

- A **refresh schedule** (re-hash + re-upload) in place so the data store stays current (Gate D).
- No-secrets and safe-by-default scripts verified; change log and data-owner/privacy sign-offs captured.

Boundary: *the solution delivers the technical configuration and the evidence. *Which* records are in scope, the* ***enforced action****, and the* ***minimisation/retention of the loaded data*** *are* ***compliance, privacy, and legal determinations****, captured alongside the delivered evidence.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PURVEDM-Discovery.ps1

Purpose: Reports the tenant's Exact Data Match (EDM) readiness: existing EDM schemas (data stores) and EDM-based sensitive information types, and — via Microsoft Graph — whether the required 'EDM_DataUploaders' security group exists and who is in it (the accounts authorized to run the EDM Upload Agent). Read-only: it changes nothing and reads no sensitive record data. HOW TO RUN Full readiness check (S&C PowerShell + Graph): powershell -File .\PURVEDM-Discovery.ps1 Skip the Graph/group check (S&C PowerShell only): powershell -File .\PURVEDM-Discovery.ps1 -SkipGraph Requires Security & Compliance PowerShell (Connect-IPPSSession) and, for the group check, Microsoft Graph (Connect-MgGraph with Group.Read.All / GroupMember.Read.All). EXPECTED RESULTS Prints the EDM schemas and EDM SITs found, and PASS/ATTENTION for the EDM_DataUploaders group (exists + has members = PASS). No schemas/SITs yet is expected on a fresh tenant. .NOTES Purview — Exact Data Match. Read-only. No credentials, salts, or record data in code. > [CmdletBinding()] param([string]$UploaderGroupName = 'EDM_DataUploaders', [string]$UserPrincipalName, [switch]$SkipGraph)

How to run: Full readiness check (S&C PowerShell + Graph): powershell -File .\PURVEDM-Discovery.ps1 Skip the Graph/group check (S&C PowerShell only): powershell -File .\PURVEDM-Discovery.ps1 -SkipGraph Requires Security & Compliance PowerShell (Connect-IPPSSession) and, for the group check, Microsoft Graph (Connect-MgGraph with Group.Read.All / GroupMember.Read.All). EXPECTED RESULTS Prints the EDM schemas and EDM SITs found, and PASS/ATTENTION for the EDM_DataUploaders group (exists + has members = PASS). No schemas/SITs yet is expected on a fresh tenant. .NOTES Purview — Exact Data Match. Read-only. No credentials, salts, or record data in code. > [CmdletBinding()] param([string]$UploaderGroupName = 'EDM_DataUploaders', [string]$UserPrincipalName, [switch]$SkipGraph)

PURVEDM-Report.ps1

Purpose: Produces a consolidated EDM report: every EDM schema (data store) and its fields, the EDM sensitive information types, which DLP policies reference each EDM SIT, and — when an EdmUploadAgent.exe path is supplied — the data store's row-count / last-upload status via the Agent's read-only /GetDataStore switch. Read-only: it reports state and changes nothing, and it reads no sensitive record data (the Agent reports hash/store metadata only). HOW TO RUN Inventory + policy references (S&C PowerShell only): powershell -File .\PURVEDM-Report.ps1 Also query data-store health via the Upload Agent: powershell -File .\PURVEDM-Report.ps1 -EdmAgentPath 'C:\Program Files\EdmUploadAgent\EdmUploadAgent.exe' -DataStoreName CustomerRecords Export to CSV: powershell -File .\PURVEDM-Report.ps1 -CsvPath .\edm-report.csv Requires Security & Compliance PowerShell (Connect-IPPSSession). Agent switches vary by version/cloud. EXPECTED RESULTS Tables of schemas, EDM SITs, and referencing DLP policies. With -EdmAgentPath, prints the Agent's /GetDataStore output (expected row count + recent upload timestamp = healthy). Stale/absent upload timestamp = ATTENTION (refresh the data — see Best Practices). .NOTES Purview — Exact Data Match. Read-only. No credentials, salts, or record data in code. > [CmdletBinding()] param([string]$EdmAgentPath, [string]$DataStoreName, [string]$CsvPath, [string]$UserPrincipalName)

How to run: Inventory + policy references (S&C PowerShell only): powershell -File .\PURVEDM-Report.ps1 Also query data-store health via the Upload Agent: powershell -File

.\PURVEDM-Report.ps1 -EdmAgentPath 'C:\Program Files\EdmUploadAgent\EdmUploadAgent.exe' -DataStoreName CustomerRecords Export to CSV: powershell -File .\PURVEDM-Report.ps1 -CsvPath .\edm-report.csv Requires Security & Compliance PowerShell (Connect-IPPSSession). Agent switches vary by version/cloud. EXPECTED RESULTS Tables of schemas, EDM SITs, and referencing DLP policies. With -EdmAgentPath, prints the Agent's /GetDataStore output (expected row count + recent upload timestamp = healthy). Stale/absent upload timestamp = ATTENTION (refresh the data — see Best Practices). .NOTES Purview — Exact Data Match. Read-only. No credentials, salts, or record data in code. > [CmdletBinding()] param([string]$EdmAgentPath, [string]$DataStoreName, [string]$CsvPath, [string]$UserPrincipalName)

PURVEDM-Schema.ps1

Purpose: Helps you author and register an EDM schema (the data-store field definition). It can EMIT a starter schema XML template (-EmitTemplate), VALIDATE an existing schema XML (well-formedness, exactly-one-or-more searchable field, field names present), and — only with -Commit — create or update the schema via New-DlpEdmSchema / Set-DlpEdmSchema. Safe by default: without -Commit it validates and prints the intended call but changes nothing. Reads no sensitive record data. HOW TO RUN 1) Emit a starter schema XML you can edit: powershell -File .\PURVEDM-Schema.ps1 -EmitTemplate -SchemaFile .\CustomerRecords-schema.xml 2) Validate your edited schema (no changes): powershell -File .\PURVEDM-Schema.ps1 -SchemaFile .\CustomerRecords-schema.xml -WhatIf 3) Create/update the schema in the tenant: powershell -File .\PURVEDM-Schema.ps1 -SchemaFile .\CustomerRecords-schema.xml -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession) for validate/commit against the tenant. EXPECTED RESULTS -EmitTemplate: writes a starter XML. -WhatIf: prints PASS/ISSUES for the XML and the New-/Set-DlpEdmSchema call it WOULD run. -Commit: creates/updates the schema and re-reads it (schema listed by Get-DlpEdmSchema = PASS). Verify with PURVEDM-Discovery.ps1. .NOTES Purview — Exact Data Match. Writes gated behind -Commit. No credentials, salts, or record data in code. Field-count / searchable-field limits evolve — verify current limits on Microsoft Learn. > [CmdletBinding()] param([string]$SchemaFile, [switch]$EmitTemplate, [string]$UserPrincipalName, [switch]$Commit)

How to run: 1) Emit a starter schema XML you can edit: powershell -File .\PURVEDM-Schema.ps1 -EmitTemplate -SchemaFile .\CustomerRecords-schema.xml 2) Validate your edited schema (no changes): powershell -File .\PURVEDM-Schema.ps1 -SchemaFile .\CustomerRecords-schema.xml -WhatIf 3) Create/update the schema in the tenant: powershell -File .\PURVEDM-Schema.ps1 -SchemaFile .\CustomerRecords-schema.xml -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession) for validate/commit against the tenant. EXPECTED RESULTS -EmitTemplate: writes a starter XML. -WhatIf: prints PASS/ISSUES for the XML and the New-/Set-DlpEdmSchema call it WOULD run. -Commit: creates/updates the schema and re-reads it (schema listed by Get-DlpEdmSchema = PASS). Verify with PURVEDM-Discovery.ps1. .NOTES Purview — Exact Data Match. Writes gated behind -Commit. No credentials, salts, or record data in code. Field-count / searchable-field limits evolve — verify current limits on Microsoft Learn. > [CmdletBinding()] param([string]$SchemaFile, [switch]$EmitTemplate, [string]$UserPrincipalName, [switch]$Commit)

PURVEDM-Set-Scope.ps1

Purpose: Reports a DLP policy's current mode and included/excluded scope, and — only with -Commit — adds or removes a pilot group (e.g. SG-EDM-Pilot) from the policy's scope for the chosen workload. Safe by default: without -Commit it prints the intended Set-DlpCompliancePolicy change but applies nothing, and it NEVER moves a policy into enforce (Enable) mode — enforcement is a deliberate, separate

portal/change-control step. Also surfaces data-refresh guidance. HOW TO RUN Show the policy's current mode + scope (no changes): powershell -File .\PURVEDM-Set-Scope.ps1 -PolicyName 'DLP-EDM-CustomerRecords-Sim' Preview adding the pilot group to the Exchange scope: powershell -File .\PURVEDM-Set-Scope.ps1 -PolicyName 'DLP-EDM-CustomerRecords-Sim' `-Workload Exchange -AddGroup 'SG-EDM-Pilot' -WhatIf Apply it: powershell -File .\PURVEDM-Set-Scope.ps1 -PolicyName 'DLP-EDM-CustomerRecords-Sim'` -Workload Exchange -AddGroup 'SG-EDM-Pilot' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). EXPECTED RESULTS Prints the policy Mode + current scope. -WhatIf: prints the Set-DlpCompliancePolicy call it WOULD run. -Commit: applies the scope change and re-reads it. Refuses to modify an ENFORCING policy. .NOTES Purview — Exact Data Match. Writes gated behind -Commit; never enables enforcement. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('Exchange','SharePoint','OneDrive','Teams','Endpoint')][string]$Workload = 'Exchange', [string]$AddGroup, [string]$RemoveGroup, [string]$UserPrincipalName, [switch]$Commit)

How to run: Show the policy's current mode + scope (no changes): powershell -File .\PURVEDM-Set-Scope.ps1 -PolicyName 'DLP-EDM-CustomerRecords-Sim' Preview adding the pilot group to the Exchange scope: powershell -File .\PURVEDM-Set-Scope.ps1 -PolicyName 'DLP-EDM-CustomerRecords-Sim' `-Workload Exchange -AddGroup 'SG-EDM-Pilot' -WhatIf Apply it: powershell -File .\PURVEDM-Set-Scope.ps1 -PolicyName 'DLP-EDM-CustomerRecords-Sim'` -Workload Exchange -AddGroup 'SG-EDM-Pilot' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). EXPECTED RESULTS Prints the policy Mode + current scope. -WhatIf: prints the Set-DlpCompliancePolicy call it WOULD run. -Commit: applies the scope change and re-reads it. Refuses to modify an ENFORCING policy. .NOTES Purview — Exact Data Match. Writes gated behind -Commit; never enables enforcement. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('Exchange','SharePoint','OneDrive','Teams','Endpoint')][string]$Workload = 'Exchange', [string]$AddGroup, [string]$RemoveGroup, [string]$UserPrincipalName, [switch]$Commit)

PURVEDM-Validate.ps1

Purpose: Runs sample text through Test-DataClassification against a named EDM sensitive information type (SIT) and reports whether it matched, at what confidence. Use it to prove the two headline EDM gates: an in-data value (with a supporting element) MATCHES, and a not-in-data / unsupported value does NOT. Read-only: it evaluates text you pass in and changes nothing. IMPORTANT: pass only values you are AUTHORIZED to use for testing (a small sanctioned test set), never real production records pasted ad hoc. HOW TO RUN Test an in-data positive sample (should match): powershell -File .\PURVEDM-Validate.ps1 -SitName 'EDM - Customer Account Match' `-Text 'Account 10058822 for Smith, contact smith@contoso.com' Test a not-in-data negative sample (should NOT match): powershell -File .\PURVEDM-Validate.ps1 -SitName 'EDM - Customer Account Match'` -Text 'Account 99999999 for Nobody' -ExpectMatch:$false Test several samples from a file (one per line): powershell -File .\PURVEDM-Validate.ps1 -SitName 'EDM - Customer Account Match' -TextFile .\samples.txt Requires Security & Compliance PowerShell (Connect-IPPSSession). Allow EDM indexing time after upload. EXPECTED RESULTS Prints MATCH/NO-MATCH + confidence per sample and PASS/FAIL against -ExpectMatch (default: expect a match). A not-in-data value returning NO-MATCH is the core EDM validation (Gate B). .NOTES Purview — Exact Data Match. Read-only. Pass only sanctioned test values. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SitName, [string]$Text, [string]$TextFile, [bool]$ExpectMatch =

$true, [string]$UserPrincipalName)

How to run: Test an in-data positive sample (should match): powershell -File .\PURVEDM-Validate.ps1 -SitName 'EDM - Customer Account Match' `-Text 'Account 10058822 for Smith, contact smith@contoso.com' Test a not-in-data negative sample (should NOT match): powershell -File .\PURVEDM-Validate.ps1 -SitName 'EDM - Customer Account Match'` -Text 'Account 99999999 for Nobody' -ExpectMatch:$false Test several samples from a file (one per line): powershell -File .\PURVEDM-Validate.ps1 -SitName 'EDM - Customer Account Match' -TextFile .\samples.txt Requires Security & Compliance PowerShell (Connect-IPPSSession). Allow EDM indexing time after upload. EXPECTED RESULTS Prints MATCH/NO-MATCH + confidence per sample and PASS/FAIL against -ExpectMatch (default: expect a match). A not-in-data value returning NO-MATCH is the core EDM validation (Gate B). .NOTES Purview — Exact Data Match. Read-only. Pass only sanctioned test values. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SitName, [string]$Text, [string]$TextFile, [bool]$ExpectMatch = $true, [string]$UserPrincipalName)